# PART I. INTRODUCTION TO JAVA 3D

1. What is Java 3D?
   - Java 3D is a high-level (Object Oriented) API for building interactive 3D applications and applets
     - It enables authors to build shapes and control animation and interaction.
     - Designed for "*Write once, run anywhere*".
2. What does Java 3D do?
   - Provide a vendor-neutral, platform-independent API
     - Initially developed by Sun Microsystems until v1.5.1.
     - Now part of *JogAmp*, a set of high performance Java libraries for 3D Graphics, Multimedia and Processing.



   - Enable high level application development
     - Authors focus upon content, not rendering
     - Java 3D handles optimal rendering
   - Achieve high performance
     - Draw via OpenGL/Direct3D
     - Uses 3D graphics hardware acceleration where available
3. What do I need to use Java 3D?
   - Software:
     - Java Runtime Environment (JRE)
     - Java 3D libraries
   - (optional) A 3D graphics accelerator

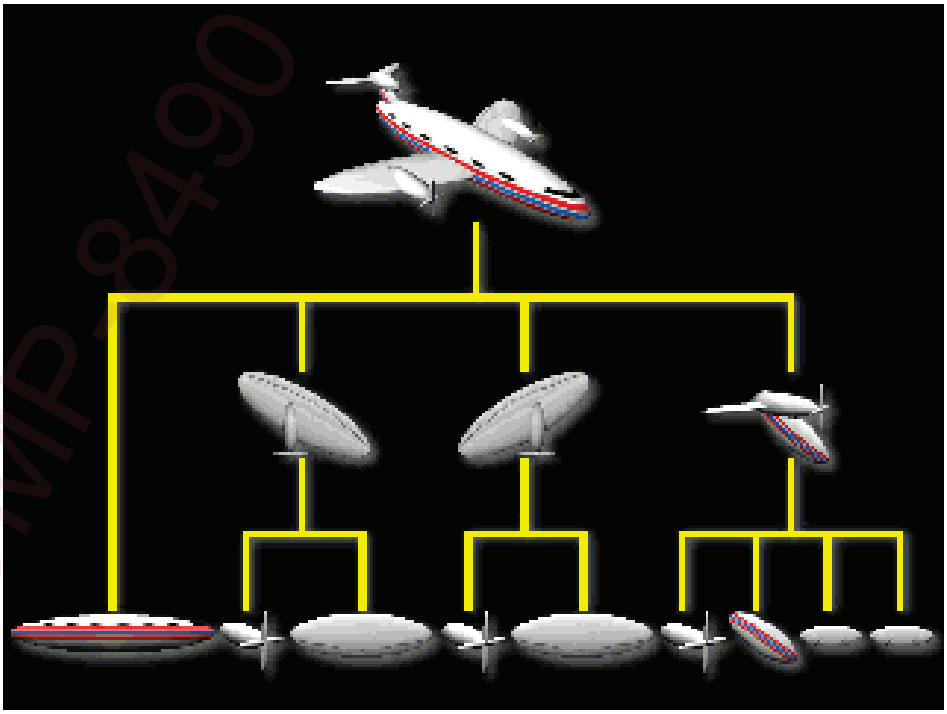
COMP8490 (xyuan@uwindsor.ca)

## PART II. BUILDING 3D CONTENT

1. A *virtual universe* holds everything.
   - A universe describes everything that we see and do within a particular world.
   - A virtual universe is a collection of scene graphs.
   - Typically there is one virtual universe per application.
2. Scene graphs
   - A *scene graph* is a hierarchical tree that describes objects and their relationship to each other.
     - "Children" are shapes, lights, sounds, etc.
     - "Parents" are groups of children and other parents

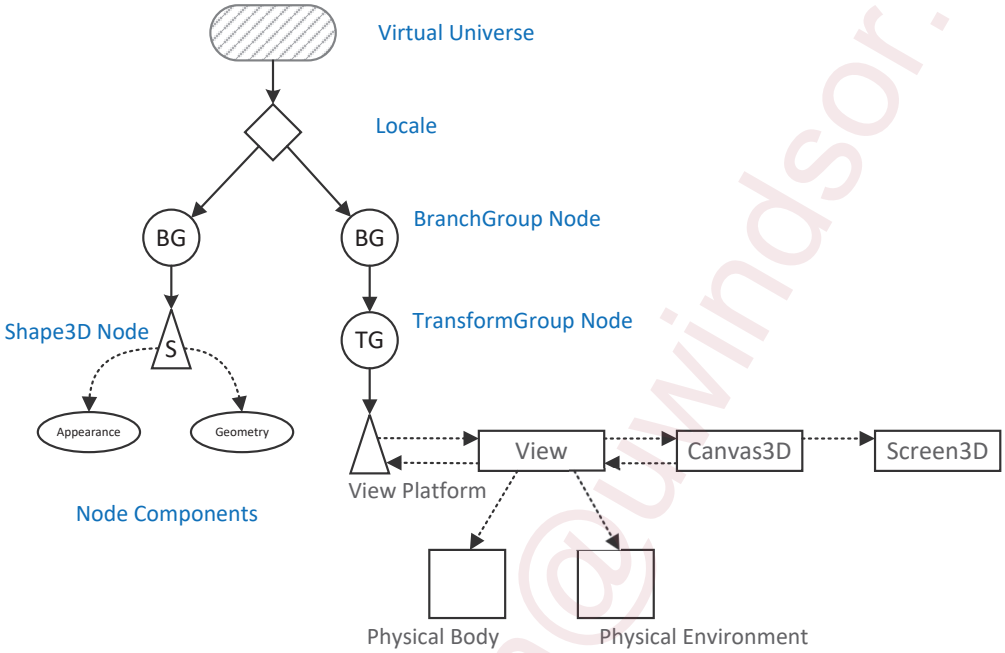


- Airplane: a scene graph example



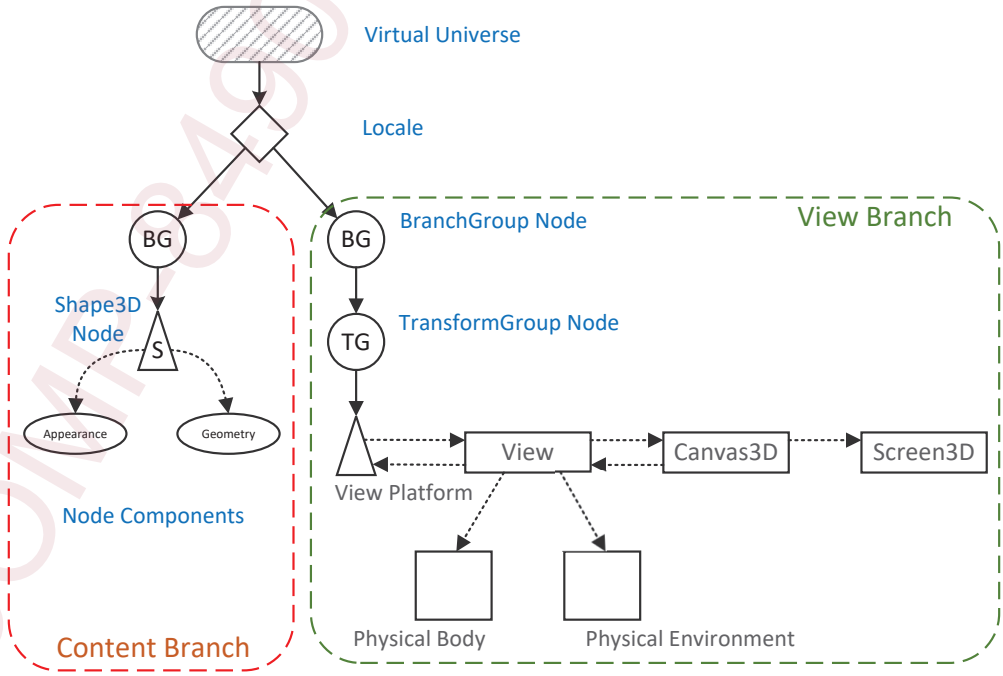
COMP 8490 (xyuan@uwindsor.ca)

- Virtual universe: a scene graph with symbols






- Scene graph: typical branches



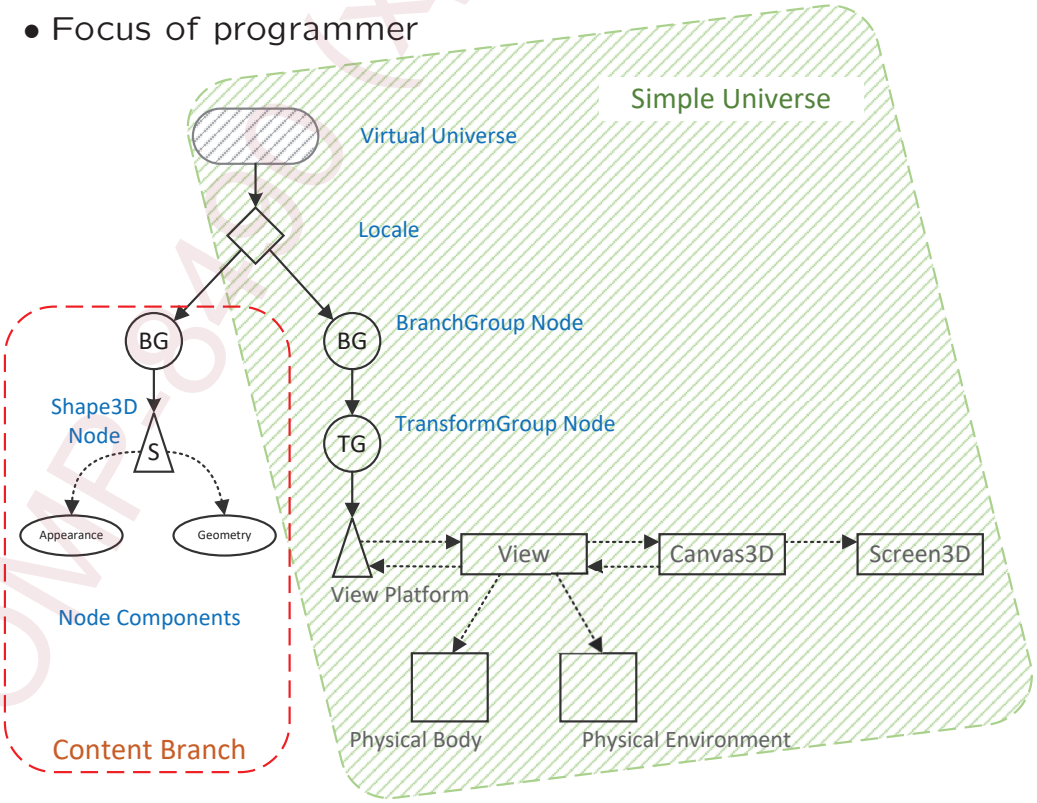

COMP3490 (xyuan@uwindsor.ca)

- A *locale* is a position in the virtual universe at which to put scene graphs.
  - Typically there is one locale per virtual universe.
  - A *branch graph* is a scene graph that forms a tree branch in the a locale of the virtual universe.
    - Typically there are several branch graphs per locale.
  - Two types of branch graphs:
    - *Content branch*: shapes, lights, and other content
      - Typically multiple content branches per locale
    - *View branch*: viewing information
      - Typically one view branch per virtual universe
  - Content and viewing information can be interleaved in the same branch (and sometimes should be)

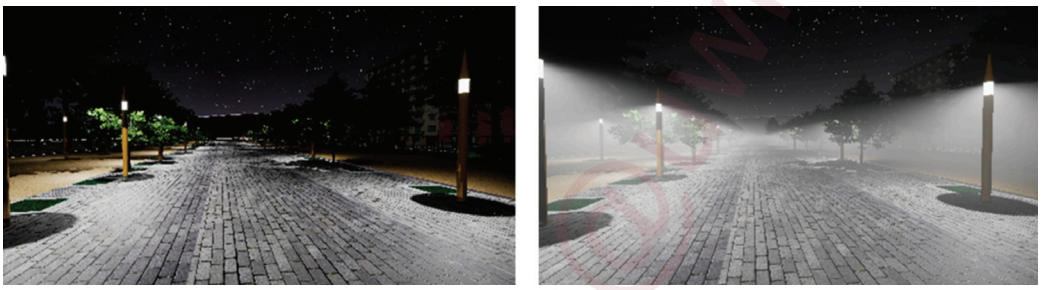
COMP 4560 (xyuan@uwindsor.ca)



- Focus of programmer



- The content branch is built from branch graphs consisting components:
  - Shapes (geometry and appearance)
  - Groups and transforms
  - Lights
  - Behaviors
  - Fog and backgrounds
  - Sounds and sound environments

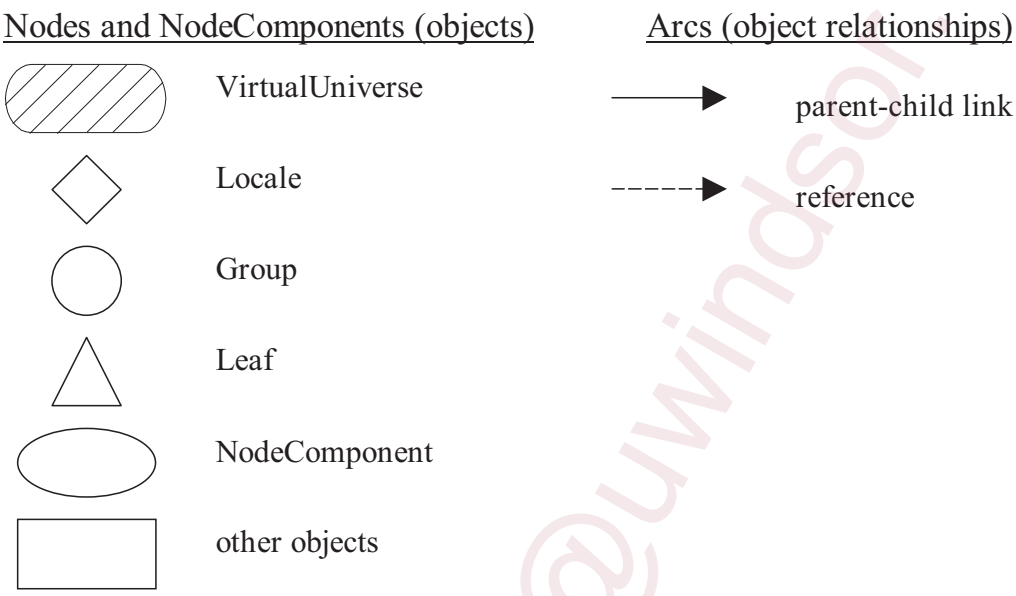




- A `SimpleUniverse` encapsulates a superstructure to build a common arrangement of a universe, locale, and viewing classes
- Processing a scene graph
  - Java 3D renders the scene graph
    - Scene graph specifies content, not rendering order
    - Rendering order is up to Java 3D
  - Java 3D uses separate, independent, and asynchronous threads
    - Graphics rendering and sound "rendering"
    - Animation "behavior execution"
    - Input device management
    - Event generation (e.g., collision detection)

COMP-8490 (xyuan@uwindsor.ca)

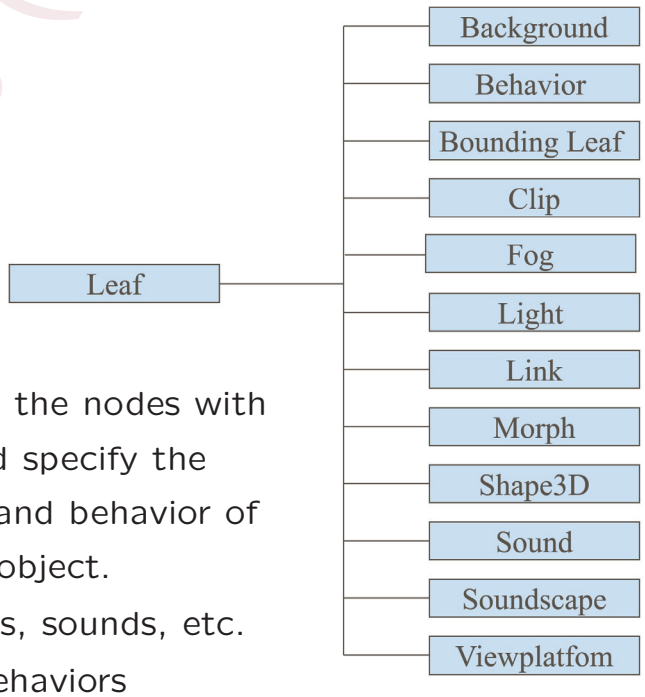
## 3. Symbols representing objects in scene graphs



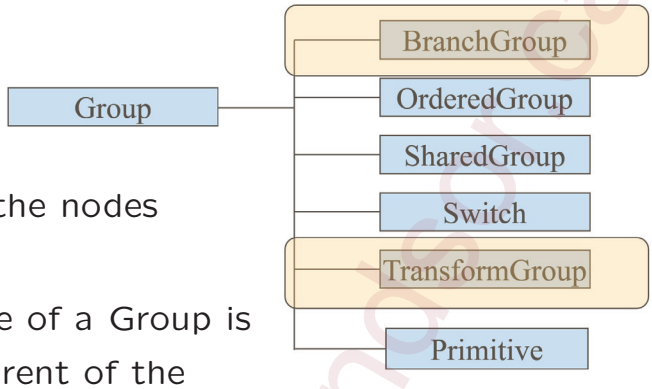

COMP-8490 (xyuan@uwindsor.ca)



- A *node* is an item in a scene graph.

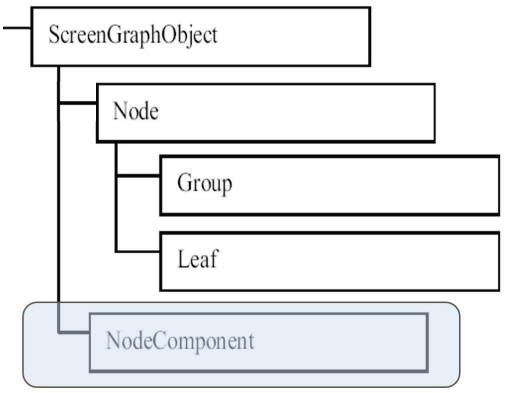



  - *Leaf nodes* are the nodes with no children and specify the shape, sound, and behavior of a scene graph object.
    - Shapes, lights, sounds, etc.
    - Animation behaviors




- *Group nodes* are the nodes with children.
  - The primary role of a Group is to act as the parent of the other nodes, specially other Group nodes and Leaf nodes.
  - Only *BranchGroup* can be children of *Locale*
  - A *TransformGroup* holds geometric transformations to specify the location and orientation of visual objects in the virtual universe.

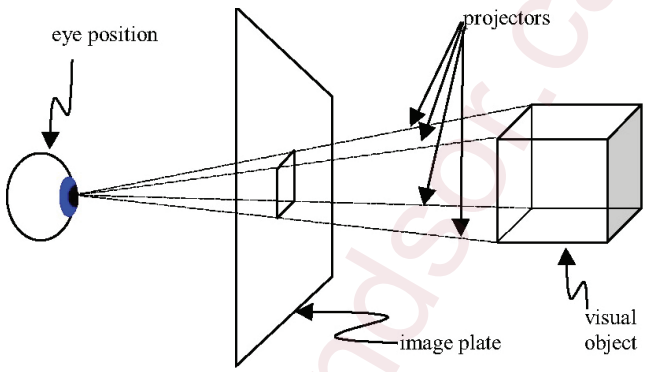





- A *node component* is a bundle of attributes used to specify the properties of *Shape3D* leaf nodes, such as:
  - Geometry of a shape
  - Texture of a shape
  - Appearance of a shape
  - Sound data to play
- Scene graphs are built from components including:
  - View platforms (viewpoints) and viewer's avatars
  - All those for constructing content branch, e.g., shapes, groups, transforms, behaviors, lights, fog, and sound

COMP-8430 (xyuan@uwindsor.ca)

4. Image plane and eye position




```
GraphicsConfiguration config = SimpleUniverse.getPreferredConfiguration();
Canvas3D canvas_3D = new Canvas3D(config);
SimpleUniverse su = new SimpleUniverse(canvas_3D); // create a SimpleUniverse
TransformGroup viewTransform = su.getViewingPlatform().getViewPlatformTransform();
Point3d eye = new Point3d(1.35, -1.35, -2.0);        // define location of the eye
Point3d center = new Point3d(0, 0, 0);               // define the point where the eye looks at
Vector3d up = new Vector3d(0, 1, 0);                 // define camera's up direction
Transform3D view_TM = new Transform3D();
view_TM.lookAt(eye, center, up);
view_TM.invert();
viewTransform.setTransform(view_TM);                 // set the TransformGroup of ViewingPlatform
```



- An image plane is the conceptual rectangle where the content of a virtual universe is projected to form the rendered image.
- Canvas3D provides an image in a window on your computer display.
  - In the real world, the devices that a canvas paints on to can be different.
    - a set of 3D shutter glasses
    - a CAVE environment, or
    - a flat monitor.
  - A graphics configuration is extracted from a system and then passed to the canvas with necessary options.

COMP-8490 (xyuan@mindspring.ca)

5. A complete view and camera representation needs to model many of the same characteristics in the 3D world.



- The `PhysicalBody` class models the body of a user, and represents how he/she is looking at the virtual world.
  - Eye position relative to the center of the head is mainly used when making stereo projections for HMDs and similar devices.
  - Ear position relative to the center of the head is used to control the projection of 3D sound.
  - Eye height from the ground is used for automatic terrain following as the rendering should match the reality to prevent motion sickness and similar physiological problems.
  - Eye position relative to the screen is used for more control over the stereo projection.

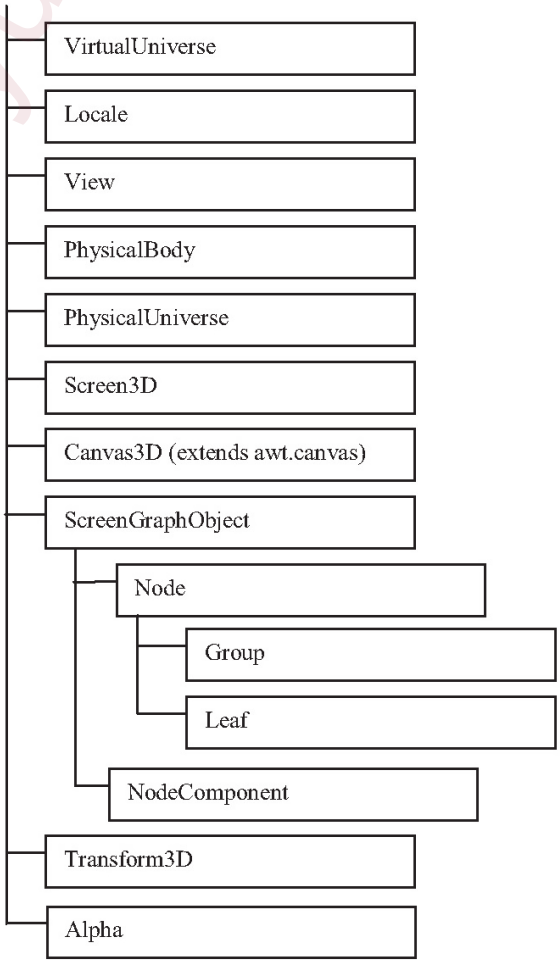
COMP-8490 (xyuan@uwindsor.ca)

- Head Tracking transform is used to control and scaling or offset calculations that need to be done.

- The `PhysicalEnvironment` class models the computer environment that the user's body sits in. It manage and installs the various devices available on your computer.
  - Audio device is installed most of the time.
  - There are also specialized devices, such as the classical Mattel PowerGlove.



6. An overview of the Java 3D API class hierarchy



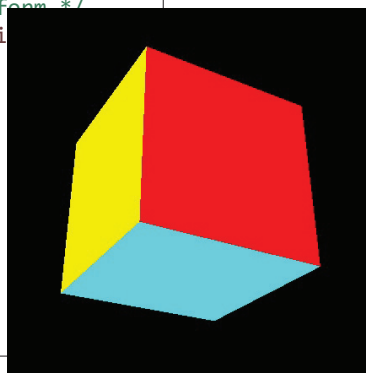

COMP-8490 (xyuan@uwindsor.ca)

## 7. Recipe for writing Java 3D programs

1. Create a Canvas3D object
2. Create a VirtualUniverse object
3. Create a Locale object, attaching it to the VirtualUniverse object
4. Construct a view branch graph
   a. Create a View object
   b. Create a ViewPlatform object
   c. Create a PhysicalBody object
   d. Create a PhysicalEnvironment object
   e. Attach ViewPlatform, PhysicalBody, PhysicalEnvironment, and Canvas3D objects to View object
5. Construct content branch graph(s)
6. Compile branch graph(s)
7. Insert subgraphs into the Locale

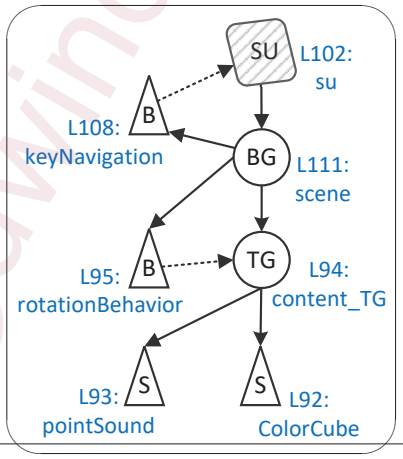


```
 3import java.applet.Applet;
17
18public class SoundCubeRotation extends Applet {
19    private static final long serialVersionUID = 1L;
20
21    /* a function to add a rotation behavior to the TransformGroup */
22    private RotationInterpolator rotateBehavior(TransformGroup my_TG) {
33
34    /* a function to define a specific location/orientation of viewer */
35    private void defineViewer(SimpleUniverse simple_U) {
46
47    /* a function to enable audio device via JOAL */
48    private void enableAudio(SimpleUniverse simple_U) {
62
63    /* a function to create and attach a PointSound to a TransformGroup */
64    private PointSound pointSound() {
77
78    /* a function to allow key navigation with the ViewingPlatform */
79    private KeyNavigatorBehavior keyNavigation(SimpleUniverse si
88
89    /* a function to build the content branch and attach to the
90    private void buildContentBranch(BranchGroup scene) {
97
98    /* a constructor to set up and run the application */
99    public SoundCubeRotation() {
117
118   /* the main entrance of the application */
119   public static void main(String[] args) {
120       new MainFrame(new SoundCubeRotation(), 600, 600);
121   }
```



COMP4900 (xyuan@uwindsor.ca)

```
89    /* a function to build the content branch and attach to the scene */
90    private void buildContentBranch(BranchGroup scene) {
91        TransformGroup content_TG = new TransformGroup();    // create a TransformGroup (TG)
92        content_TG.addChild(new ColorCube(0.4));             // attach a ColorCube to TG
93        content_TG.addChild(pointSound());                   // attach a PointSound to TG
94        scene.addChild(content_TG);                          // add TG to the scene BranchGroup
95        scene.addChild(rotateBehavior(content_TG));          // make TG continuously rotating
96    }
97
98    /* a constructor to set up and run the application */
99    public SoundCubeRotation() {
100       GraphicsConfiguration config = SimpleUniverse.getPreferredConfiguration();
101       Canvas3D canvas_3D = new Canvas3D(config);
102       SimpleUniverse su = new SimpleUniverse(canvas_3D);
103       enableAudio(su);
104       defineViewer(su);
105
106       BranchGroup scene = new BranchGroup();
107       buildContentBranch(scene);
108       scene.addChild(keyNavigation(su));
109
110       scene.compile();
111       su.addBranchGraph(scene);
112
113       setLayout(new BorderLayout());
114       add("Center", canvas_3D);
115       setVisible(true);
116   }
```






## 8. The Example

- Building the content branch
  - add a shape (L92) and a sound (L93) to a transform group (L91) and then to the branch group (L94)
  - add animation to spin the shape and sound (L95)
  - add behavior to change the view with keyboard (L108)
  - optimize the branch group for faster rendering (L110)
  - add the branch group to simple universe (or a locale) and make its nodes *live*, i.e., drawable (L111)
    - use `removeBranchGraph(aBranch)` to remove `aBranch`
- Making the virtual environment visible
  - place the 3D canvas (L101) of a simple universe (L102) in a frame (L120 and L114) before making visible (L115)

COMP 3400 (xyuan@uwindsor.ca)

## PART III. VIRTUAL REALITY and APPLICATIONS

1. Virtual Reality: The *Science* of Illusion

- Virtual Reality (VR) *creates the illusion of being in an environment that can be perceived by the user as a believable place with enough interactivity to perform specific tasks in an efficient and comfortable way.*
  - It uses computers to create 3D environments in which the user can navigate and interact *in real time*.
- Two Main Factors of VR Experience
  - Immersion refers to the physical configuration of the user interface of the VR application.
  - Presence is a state of consciousness, the (psychological) sense of being in the virtual environment.

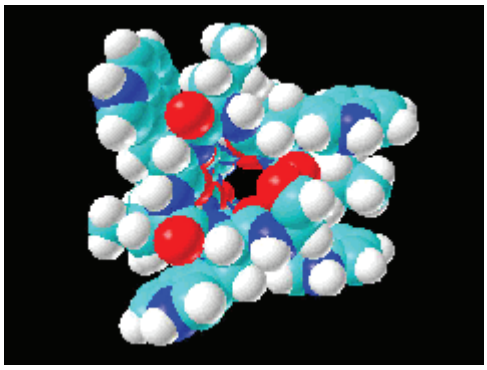


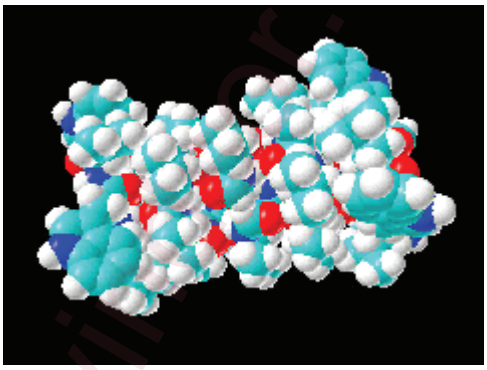
2. Typical areas of applications

- Scientific visualization
- Information visualization
- Medical visualization
- Geographical information systems (GIS)
- Computer-aided design (CAD)
- Animation
- Education
- E-Commerce
- Games

COMP-8490 (xyuan@uwindsor.ca)

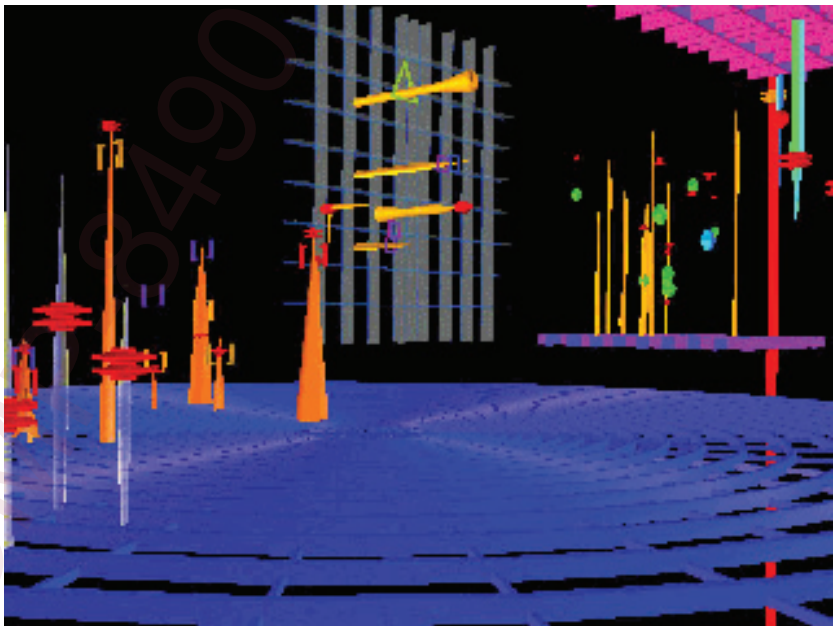
- Examples: Scientific Visualization





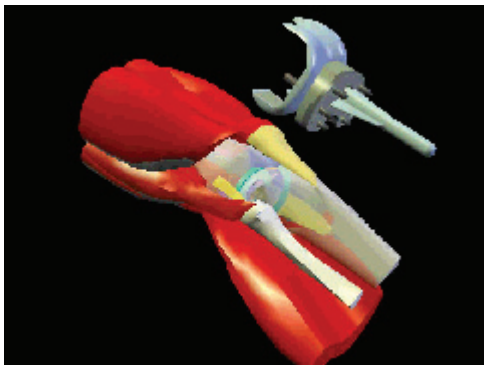


- Examples: Abstract Data (Financial)



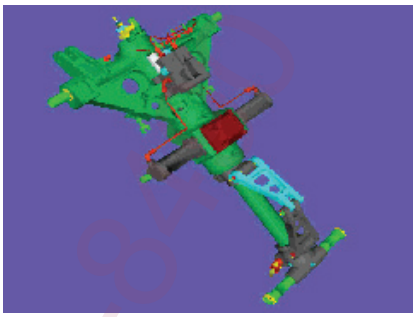
COMP3490 (xyuan@uwindsor.ca)

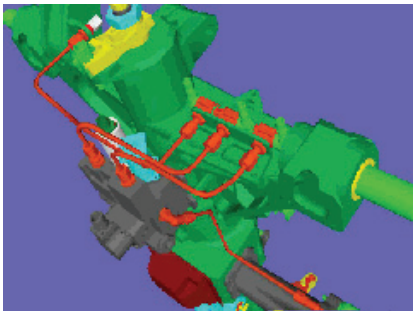
- Examples: Medical Education





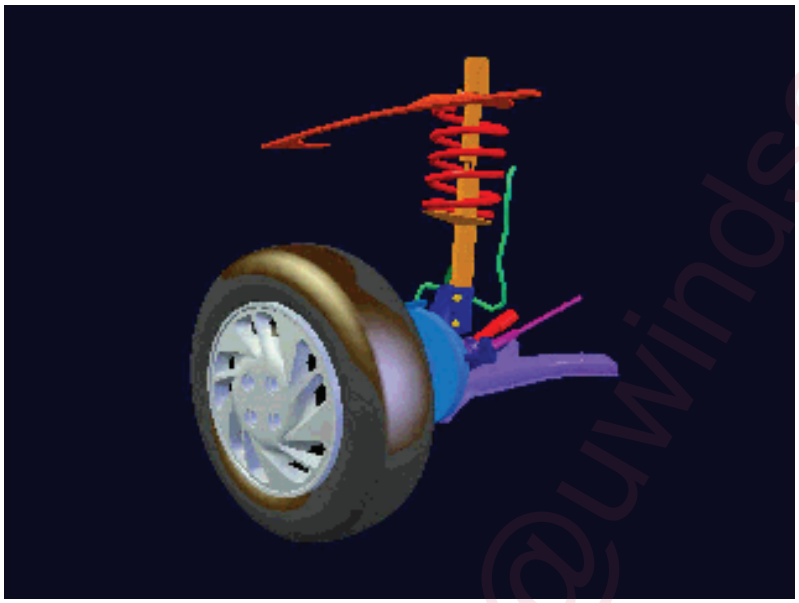
- Examples: CAD





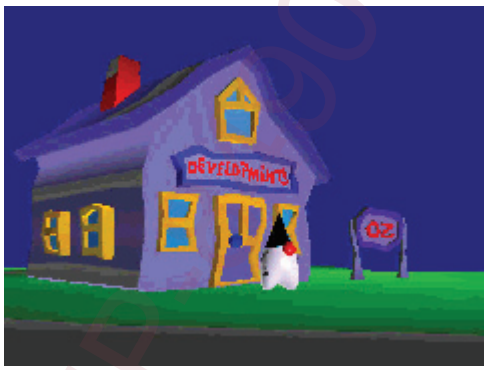
COMP-8421 (xyuan@uwindsor.ca)

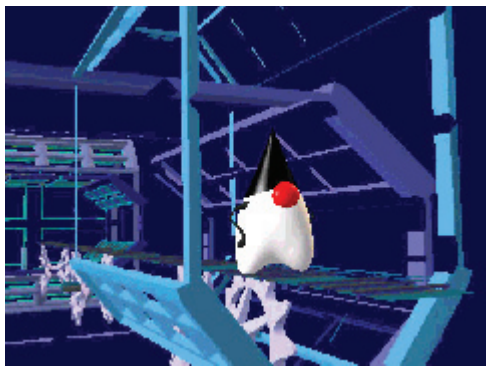
- Examples: Mechanical Analysis





- Examples: Computer Animations



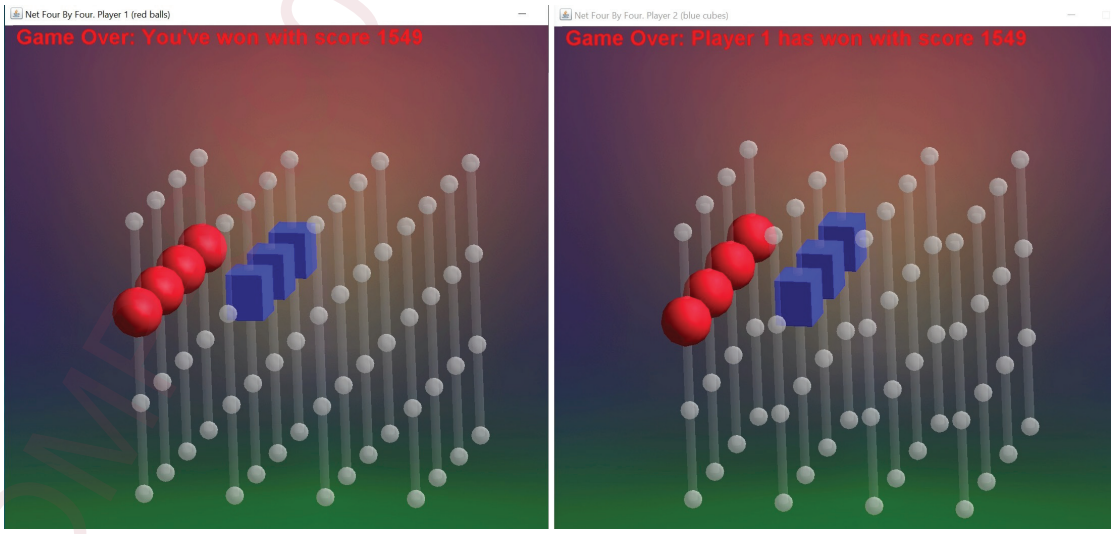



COMP 390 (xyuan@uwindsor.ca)

- Examples: E-Commerce



- Examples: Computer Games




COMP 3490 (xyuan@mun.ca)